

ThreatSense AI

AI-Powered Threat Intelligence Correlation & Alert Prioritisation Assistant

IBM BoB AI Innovation Hackathon 2026

Problem D2 — Threat Intelligence Correlation & Alert Prioritisation Assistant

Track: AI

Team: CyberGuard

1. Overview

ThreatSense AI is an AI-powered threat intelligence correlation and alert prioritisation assistant designed for analysts who need to process large numbers of security alerts. The system correlates related alerts, assigns a 0–100 risk score, maps activity to MITRE ATT&CK techniques, and generates a concise BLUF (Bottom Line Up Front) summary for investigation.

The project is implemented as a full-stack application with a FastAPI backend and React frontend. IBM watsonx.ai Granite is used for BLUF generation when configured, with a rule-based fallback for demo use.

2. Getting Started — Use the Submission

The project is structured to follow the faculty-provided Bobathon submission requirements. The repository should remain public and should contain the required submission metadata, documentation, source code, demo evidence, presentation, and validation workflow.

1. Confirm the repository is public and named according to the Bobathon convention.
2. Complete all team/member details in submission.yaml and README.md.
3. Verify the documentation files in docs/.
4. Run the backend and frontend locally and confirm the application works.
5. Add the actual demo video, live-demo status, and at least three screenshots.
6. Check GitHub Actions and confirm Validate Submission is green.
7. Submit the repository URL through the official hackathon entry form.

3. Repository Structure

```
bob-ai-hackathon-cyberguard/  
├── submission.yaml  
├── README.md  
├── src/  
│   ├── backend/  
│   │   ├── main.py  
│   │   ├── alerts.py  
│   │   ├── correlator.py  
│   │   ├── analyzer.py  
│   │   ├── watsonx_client.py  
│   │   └── requirements.txt  
│   └── frontend/  
│       └── App.tsx
```

```

├── components/
├── types/
├── package.json
├── vite.config.ts
├── .env.example
├── docs/
│   ├── problem-statement.md
│   ├── solution-overview.md
│   ├── architecture.md
│   └── setup-guide.md
├── demo/
│   ├── demo-video-link.txt
│   ├── live-demo-url.txt
│   └── screenshots/
├── presentation/
│   └── slides.pdf or slides.pptx
├── CONTRIBUTING.md
├── .gitignore
├── .github/
│   └── workflows/
│       └── validate.yml

```

4. File-by-File Documentation

4.1 submission.yaml — Evaluator Metadata

submission.yaml is the evaluator-facing metadata file. It should contain the final team information, project title, selected problem statement, solution summary, and implemented key features.

```

team:
  name: "CyberGuard"
  track: "AI"
  lead:
    name: "[ACTUAL TEAM LEAD NAME]"
    email: "[ACTUAL INSTITUTIONAL EMAIL]"
  members:
    - name: "[MEMBER 1]"
      email: "[EMAIL]"
    - name: "[MEMBER 2]"
      email: "[EMAIL]"
    - name: "[MEMBER 3]"
      email: "[EMAIL]"

submission:
  title: "ThreatSense AI"
  problem_statement: >
    Defence analysts receive large volumes of alerts from SIEM systems,
    cyber sensors, intelligence reports and other sources, making it
    difficult to identify genuine threats and prioritise investigations.
  solution_summary: >
    ThreatSense AI correlates related alerts, calculates a 0-100 risk score,
    maps threats to MITRE ATT&CK techniques, and generates concise BLUF
    investigation summaries using IBM watsonx.ai Granite.
  key_features:
    - "AI-driven alert correlation by source IP, attack pattern and time window"
    - "0-100 risk scoring using severity, frequency, asset criticality and MITRE coverage"
    - "MITRE ATT&CK technique mapping"

```

- "BLUF threat summaries using IBM watsonx.ai Granite"
- "Interactive SOC dashboard with filtering and alert classification"

Important: replace every placeholder above with the actual team information before submission. Do not submit placeholder names or example email addresses.

4.2 README.md

The README should act as the human-readable front page of the repository.

Team

Team: CyberGuard

Track: AI

Project: ThreatSense AI

Problem Statement

Defence and security analysts receive thousands of alerts from multiple sources. Manually correlating these alerts, distinguishing genuine threats from false positives, mapping attacker behaviour to MITRE ATT&CK, and preparing investigation summaries can delay response.

Solution

ThreatSense AI provides a unified analyst interface that groups related alerts, calculates a risk score, identifies relevant MITRE ATT&CK techniques, and produces a concise BLUF summary. The application combines deterministic analysis with IBM watsonx.ai Granite for natural-language summarisation.

Key Features

- Alert correlation based on source IP, attack pattern and time window.
- Risk score from 0–100 using severity, frequency, asset criticality and MITRE ATT&CK coverage.
- MITRE ATT&CK technique identification including examples such as T1059, T1190 and T1078.
- BLUF summaries generated through IBM watsonx.ai Granite when configured.
- Interactive dashboard for severity, status and source filtering, alert details and classification.

Technology Stack

- Python 3.11+
- FastAPI and Uvicorn
- React 18, TypeScript, Vite and Tailwind CSS
- IBM watsonx.ai Granite-3-8B-Instruct
- IBM Bob AI Assistant
- Simulated SIEM alert data for the current demonstration

How to Run

Backend

```
cd src/backend
pip install -r requirements.txt
cp ../.env.example ../.env
uvicorn main:app --reload --port 8000
```

Frontend

```
cd src/frontend
npm install
npm run dev
```

The frontend runs on the Vite development server, normally at <http://localhost:5173>, while the FastAPI backend runs on port 8000.

Demo

Add the final demo video URL, live-demo status, and screenshots before submission. The video should demonstrate application startup, a real analyst workflow, and actual generated output.

Known Limitations

- The current alert dataset is simulated.
- watsonx.ai BLUF generation requires the appropriate IBM Cloud/API configuration; a fallback is available.
- Correlation currently uses heuristic time-window and IP-overlap logic rather than trained ML clustering.
- Authentication and rate limiting are not currently implemented.

What We Are Most Proud Of

The strongest part of ThreatSense AI is the end-to-end analyst workflow: alert ingestion, correlation, risk prioritisation, MITRE ATT&CK mapping, AI-assisted BLUF generation, and interactive investigation are brought together in one application.

4.3 docs/

[docs/problem-statement.md](#)

Threat intelligence teams and defence analysts must process high volumes of security alerts from heterogeneous sources. The main difficulty is not simply receiving alerts, but determining which alerts belong to the same incident, which events represent meaningful risk, and which investigation should be handled first.

Existing workflows often require analysts to manually pivot between alert records, source information, attack-pattern information and threat-intelligence references. This creates alert fatigue and can increase the time required to understand an incident.

Problem D2 focuses on ingesting multi-source feeds, correlating alerts, separating genuine threats from false positives, mapping attacker techniques to MITRE ATT&CK, and generating prioritised BLUF summaries.

[docs/solution-overview.md](#)

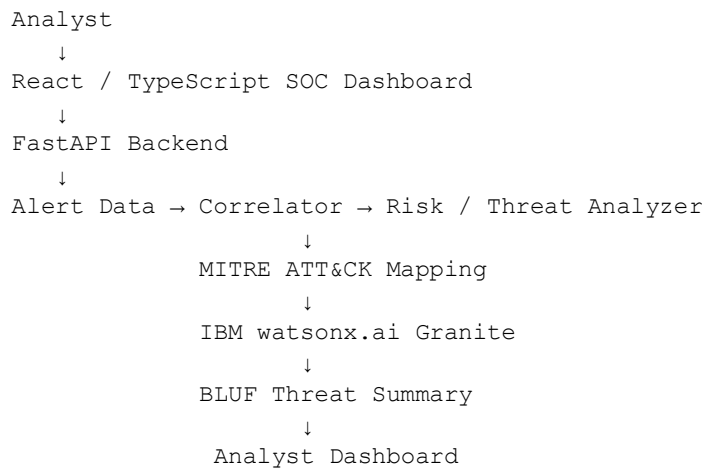
ThreatSense AI uses a staged analysis pipeline. Incoming alert data is first normalised and correlated. The analysis layer then combines alert severity, event frequency, asset criticality and MITRE ATT&CK coverage to calculate a 0–100 risk score. Relevant techniques are surfaced for analyst context.

For the highest-value analyst output, the system can use IBM watsonx.ai Granite to generate a concise BLUF summary. If the watsonx.ai credentials are unavailable, the application can use its rule-based fallback so the demonstration remains functional.

The key design decision is to combine transparent rule-based security analysis with AI-generated language. This allows the risk and correlation logic to remain understandable while using generative AI where it adds the most value: summarising the investigation context.

[docs/architecture.md](#)

End-to-end flow:



Technology	Responsibility	Role in ThreatSense AI
React / TypeScript	User interface	SOC dashboard and analyst interaction
FastAPI	Backend API	Serves alert and analysis functionality
Python	Analysis	Correlation, scoring and threat analysis
IBM watsonx.ai Granite	Generative AI	BLUF threat summarisation
MITRE ATT&CK	Threat framework	Technique mapping and analyst context
Simulated SIEM data	Input source	Demonstration alert stream

Security and scalability notes: real credentials should remain outside the repository. The current implementation is intended as a hackathon demonstration and does not yet include authentication, rate limiting, or production-scale SIEM ingestion.

[docs/setup-guide.md](#)

Prerequisites

- Python 3.11 or later
- Node.js and npm
- A public GitHub repository for submission
- IBM watsonx.ai credentials if AI-generated BLUF output is required

Installation and execution:

8. Clone the repository.
9. Open a terminal in src/backend.
10. Install Python dependencies using `pip install -r requirements.txt`.
11. Create the environment file from `.env.example`.
12. Start FastAPI using `uvicorn main:app --reload --port 8000`.
13. Open another terminal in src/frontend.
14. Run `npm install`.
15. Run `npm run dev`.
16. Open the displayed Vite URL and verify that alerts and analysis are shown.

Troubleshooting

Problem	Action
Backend does not start	Confirm Python version and install

	requirements.txt.
Frontend does not start	Run npm install inside src/frontend and retry npm run dev.
AI summary unavailable	Check IBM watsonx.ai configuration; the fallback mode can still support the demo.
Port already in use	Stop the process using the port or run the service on an available port.

4.4 src/

All application source code is contained under src/. The backend contains the API and threat-analysis logic, while the frontend contains the React dashboard and user-interface components.

Environment variables should be documented in .env.example. Real credentials must never be committed.

4.5 demo/

The demo folder provides evidence that the application works. The final submission should contain:

- demo/demo-video-link.txt with the actual 3–5 minute demo URL.
- demo/live-demo-url.txt containing the deployed URL or NOT DEPLOYED.
- At least three screenshots of the running application.

Recommended screenshot sequence:

17. 01-home-dashboard.png — main SOC dashboard
18. 02-alert-analysis.png — selected alert and analysis
19. 03-threat-summary.png — prioritisation / BLUF output

4.6 presentation/

The presentation should cover the problem, solution, demo/architecture, IBM technology integration, and expected impact. The IBM Bob and watsonx.ai roles should be clearly demonstrated rather than merely mentioned.

5. Automated Validation

The repository should retain the provided .github/workflows/validate.yml workflow. After pushing changes, check the GitHub Actions tab and confirm that Validate Submission is green.

- submission.yaml exists and is valid YAML.
- Required metadata fields are completed.
- Required documentation files are present.
- Demo link file is present.
- The repository remains structurally complete.

6. Submission Checklist

- submission.yaml — all required fields filled
- README.md — no placeholder text remains
- docs/problem-statement.md — completed
- docs/solution-overview.md — completed
- docs/architecture.md — diagram and explanation present

- docs/setup-guide.md — tested end-to-end
- src/ — all source code committed and .env.example updated
- demo/demo-video-link.txt — real working video URL
- demo/live-demo-url.txt — deployed URL or NOT DEPLOYED
- demo/screenshots/ — at least three running-application screenshots
- presentation/slides.pdf or slides.pptx — present
- No .env, node_modules, .venv or build artifacts committed
- Validate Submission GitHub Action is green
- Repository is Public
- Entry form is submitted before the deadline

7. How the Entry Is Evaluated

Criterion	Points	What ThreatSense AI Should Demonstrate
Technical Implementation Quality	25	Working source code, meaningful backend/frontend implementation and technical quality
Innovation & Differentiation	25	Useful correlation, prioritisation and AI-assisted investigation workflow
Problem Depth & Vision	15	Clear understanding of analyst alert overload and threat-intelligence workflow
Working Demo & Functionality	15	Reproducible application and convincing end-to-end demo
IBM Bob Integration	10	Genuine use of IBM Bob in development and solution workflow
Documentation & Reproducibility	10	Clear setup, architecture and reproducible project documentation

8. Common Mistakes to Avoid

- Leaving placeholder team/member information in submission.yaml or README.md.
- Committing real API keys or .env files.
- Leaving the demo-video-link.txt with an example URL.
- Keeping the repository private.
- Submitting a source directory containing only boilerplate.
- Providing setup instructions that have not been tested.
- Describing IBM technologies without showing how they contribute to the solution.
- Claiming trained machine-learning correlation when the current implementation uses heuristic correlation.

9. Final Submission Notes

ThreatSense AI should be submitted as an implementation-focused hackathon project rather than only as a concept. The final repository should demonstrate a working analyst workflow, clear IBM technology integration, reproducible setup instructions, and honest documentation of current limitations.

Before submission, replace all team placeholders with the actual CyberGuard member names and institutional email addresses, add the real demo evidence, verify the repository is public, and confirm the validation workflow is green.